

## Review of Union-Find data type

A *union-find* (or *disjoint-set*) data type manages a collection of disjoint sets over a universe of elements named  $0, 1, \dots, n - 1$ , with each element in exactly one set. Every set designates one of its elements as a *leader* (or *root*) that uniquely identifies the set. The two essential operations are:

- `int find(p)` — return the leader of the set containing  $p$ .
- `void unify(p, q)` — merge the set containing  $p$  with the set containing  $q$ .

The main use case is answering *connectivity queries*: are  $p$  and  $q$  in the same set? (i.e. `find(p) = find(q)`). Disjoint sets model many situations: connected components of a graph, clusters of conducting sites, inter-linked friends in a social network, equivalent variable names, and — as in this lab — **adjoining stones of the same color in Hex**.

### Quick-Find

Store an array `leader[]` of length  $n$ , where `leader[i]` is the leader of the set containing element  $i$ . Initially `leader[i] = i` for all  $i$ .

- `find(p)`: return `leader[p]` — one array access.
- `unify(p, q)`: change every array entry whose value equals `leader[p]` to `leader[q]` (or vice versa) — at least  $n$  array accesses.

The cost of `unify` is too high: processing  $m$  operations on  $n$  elements takes  $mn$  array accesses — quadratic in the input size.

### Quick-Union

Represent each set as a rooted tree in a forest: an array `parent[]` where `parent[i]` is the parent of element  $i$  (a root points to itself). A set's leader is the root of its tree.

- `find(p)`: chase parent pointers until reaching the root — time proportional to the depth of  $p$ .
- `unify(p, q)`: set `parent[root(p)] = root(q)` — only one array entry changes.

If trees become tall, `find` becomes expensive: in the worst case the depth reaches  $n - 1$ , so  $m$  operations again take  $mn$  time.

### Weighted Quick-Union (link-by-size)

Keep an extra array `size[i]` counting the number of elements in the tree rooted at  $i$ , initially 1. Always attach the root of the *smaller* tree to the root of the *larger* tree, and update `size[]`.

- `find(p)`: unchanged from quick-union.

- **unify**(*p*, *q*): link the smaller root under the larger root.

Link-by-size guarantees that the depth of any node is at most  $\log_2 n$  (each time a node's depth increases, the size of its tree at least doubles).

## Path Compression

Flatten the tree during **find**: while searching for the root of *p*, make every visited node point directly to the root (**parent**[*i*] = **find**(**parent**[*i*])). Combined with weighted quick-union, the amortized cost of each operation drops to essentially constant:  $\mathcal{O}(\alpha(m, n))$ , where  $\alpha$  is the extremely slow-growing *inverse Ackermann function*.

## Summary

For  $m \geq n$  union-find operations on *n* elements:

| Algorithm                               | worst-case time  |
|-----------------------------------------|------------------|
| Quick-Find                              | $mn$             |
| Quick-Union                             | $mn$             |
| Weighted Quick-Union                    | $m \log n$       |
| Weighted Quick-Union + path compression | $m \alpha(m, n)$ |

The difference is not academic:  $10^9$  unions and finds on  $10^9$  elements take about 30 years with quick-find but only about 6 seconds with weighted quick-union plus path compression.

## Hex (board game)

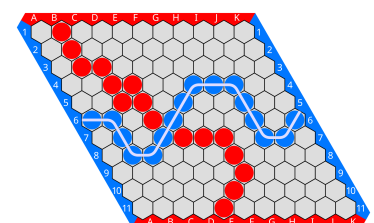
In this lab we put the union-find data type to work on a classic board game. *Hex* is a two-player board game in which players attempt to connect opposite sides of a rhombus-shaped board made of hexagonal cells. Hex was invented by mathematician and poet Piet Hein in 1942 and later rediscovered and popularized by John Nash.

It is traditionally played on an  $11 \times 11$  rhombus board. The board is composed of hexagons called *cells* or *hexes*. Each player is assigned a pair of opposite sides of the board, which they must try to connect by alternately placing a stone of their color onto any empty hex. Once placed, the stones are never moved or removed. A player wins when they successfully connect their sides together through a chain of adjacent stones. Draws are impossible in Hex due to the topology of the game board.

To check whether a player has won, we will use Union-Find data type. We will model the board as a set of elements, use virtual boundary nodes for the sides, and let **unify**() merge stones of the same color into connected components.

You can play the game yourself against the computer at <https://playabstractgames.com/en/hex/play/computer/> before we implement it.

**Board model.** The board is an  $n \times n$  rhombus of hexagonal cells. Index the cells by row *r* and column *c*, both from 0 to  $n-1$ . Two cells are *neighbors* when they share an edge; on the rhombus board each cell has up to six neighbors, in axial coordinates  $(r \pm 1, c)$ ,  $(r, c \pm 1)$ ,  $(r-1, c+1)$ ,  $(r+1, c-1)$ .



**Winning.** Player 0 (Red) wins by connecting the *top* side ( $r = 0$ ) to the *bottom* side ( $r = n - 1$ ) through a chain of adjacent Red stones. Player 1 (Blue) wins by connecting the *left* side ( $c = 0$ ) to the *right* side ( $c = n - 1$ ) through a chain of adjacent Blue stones. *Draws are impossible:* on a completely filled board exactly one player must have a winning path. This is the **No-Draw Theorem**.

## Hex as a Union-Find Problem

The key idea is to let Union-Find maintain the connected components of stones. We introduce four *virtual* nodes — one for each of the four sides to connect:

$$\text{TOP} = n^2, \quad \text{BOTTOM} = n^2 + 1, \quad \text{LEFT} = n^2 + 2, \quad \text{RIGHT} = n^2 + 3.$$

A stone of player 0 placed on the top row is unioned with TOP; on the bottom row it is unioned with BOTTOM. A stone of player 1 on the left column is unioned with LEFT; on the right column with RIGHT. Every placed stone is unioned with its already-placed same-color neighbors. A player wins the moment their two virtual boundary nodes fall into the same component, i.e. `find(virtualA) = find(virtualB)`.

### Part A: Complete the Union-Find Class

Use the code in `union-find.hpp` as a starting point. The class is already initialized with the correct number of elements ( $n^2 + 4$ ) and the correct initial state (every element its own parent, size 1). Complete the two methods:

- `int find(int i)` — return the root of the set containing `i`. Use **path compression**: while searching, make every node point directly to the root (recursively or iteratively).
- `void unify(int p, int q)` — merge the sets containing `p` and `q`. Use **union by size**: attach the root of the smaller tree under the root of the larger tree.

Handle the edge case where `p` and `q` are already in the same set (do nothing).

### Part B: The Hex Board on Top of Union-Find

Use the code in `hex-board.hpp` as a starting point. It is a template `HexBoard<UF>` that wraps any Union-Find type. Complete:

- `bool place(int r, int c, int player)` — mark the cell empty (handle the already-occupied case), union it with every in-bounds same-color neighbor, union it with the appropriate virtual boundary node, and return `true` if the move completed a winning path for `player`.
- `bool isWinner(int player)` — return whether the two virtual boundary nodes of `player` are connected.
- `bool isFull()` — return whether every cell is occupied.

### Part C: The No-Draw Theorem

The provided `main.cpp` fills  $n \times n$  boards completely with random stones and checks, over many trials, that **exactly one** player wins — never both, never neither. This is the mathematical fact that makes Hex a game with no draws. Compile and run the program and confirm the theorem holds for every trial.

## Part D: Performance — Quick-Find vs. Quick-Union vs. Weighted Quick Union

The `main.cpp` also times the *same* random sequence of moves on three different Union-Find backends:

- `QuickFindUF` —  $O(n)$  per `find/unify`
- `QuickUnionUF` —  $O(n)$  worst case per operation
- `UF` (Weighted Quick Union with path compression) — effectively  $O(\alpha(n))$  per operation

Because the identical random sequence (fixed seed) is replayed on each backend, the comparison is fair. Compile and run the program, observe the timings, and write a short report `report.txt` explaining why the weighted version is dramatically faster on the large board.

### Submission

Submit the following files:

- `union-find.hpp`: complete Union-Find class (path compression + union by size).
- `hex-board.hpp`: complete Hex board class.
- `main.cpp`: your test harness (do not modify unless needed).
- `report.txt`: a short report on the No-Draw Theorem and the performance comparison of the three Union-Find implementations.